

F1 Car Detection and Team Classification

A Comparative Study of YOLOv8 and SSD Architectures

Mohamed Elsayed

Abstract

This report presents a comparative study of two object detection architectures, YOLOv8 and SSD (Single Shot MultiBox Detector), applied to the task of detecting and classifying Formula 1 racing cars by team from video footage. The motivation behind this project came from the increasing use of computer vision in sports broadcasting, where automated team identification could enhance viewer experience and statistical analysis. A publicly available dataset from Roboflow containing over 6,600 labelled images across four classes (Ferrari, McLaren, Mercedes, and Red Bull) was used for training and evaluation. Both models were trained under four different configurations each, varying parameters such as batch size, image resolution, and learning rate. Results show that YOLOv8 significantly outperforms SSD across all metrics, achieving an average mAP50 of 0.9577 compared to SSD's 0.9117. The best performing configuration overall was YOLOv8 Config C, which reached an mAP50 of 0.9608. Additionally, the trained YOLOv8 model was tested on real F1 broadcast footage and successfully detected and classified cars from all four teams in real time.

Introduction

Formula 1 has always been a sport where data matters. Teams spend millions analysing every aspect of car performance, and the broadcast side is always trying to find new ways to make the experience more engaging. One thing that caught my attention was whether it would be possible to identify which team a car belongs to just from the camera footage, without needing any external signals or manual labelling.

The goal of this project is to find out whether a deep learning model can do this reliably. It sounds straightforward at first since the teams do have quite different colours, but in practice there are complications. Cars can be partially hidden, far away, blurred from motion, or shot from awkward angles. The four teams covered are Ferrari (red), McLaren (orange), Mercedes (silver), and Red Bull (dark blue with yellow). The silver of Mercedes in particular can be tricky against certain backgrounds.

Two models were picked for comparison: YOLOv8, which is currently one of the most popular real-time detectors, and SSD with a VGG16 backbone, which is older but widely used as a baseline. The aim is not just to find which one does better, but also to understand how changing things like the batch size or image resolution affects each model differently, and what that tells us about deploying these models in a real broadcast setting.

The report is organised as follows: Section 2 covers some background on object detection. Section 3 explains the dataset and how the experiments were set up. Section 4 goes through the results for both models. Section 5 discusses what the findings mean, and Section 6 wraps things up.

Related Work

Object detection has come a long way over the last ten years or so. The older methods relied on things like HOG features and sliding windows, which were slow and did not handle different object sizes very well. Deep learning changed all of that.

R-CNN (Girshick et al., 2014) was an early breakthrough that used a separate region proposal step before classifying each region. The problem was it was too slow for anything real-time. SSD and YOLO (both 2016)

came along and showed you could do the whole thing in one pass through the network, which made detection fast enough for video applications.

YOLOv8 came out in 2023 and is the latest version from Ultralytics. It dropped the traditional anchor boxes in favour of a cleaner anchor-free approach, added a new C2f module that helps with training, and separated the classification and box prediction into different branches. These changes made it noticeably better than YOLOv5 on standard benchmarks.

SSD is still commonly used, especially in situations where you need something that runs on limited hardware. It is well documented and easy to implement, which is partly why it was chosen here as the comparison model.

Computer vision has been applied in sports quite a bit already, mainly for player tracking and ball detection. Kamble et al. (2019) used YOLO for cricket ball tracking, and there has been work on identifying football players. Doing this for F1 specifically is less common in the literature, which made it feel like a worthwhile angle to explore.

Methodology

Dataset

The dataset was taken from Roboflow Universe, specifically the F1 Car Classification dataset version 3. It contains images from F1 race broadcasts and official photos, with bounding boxes drawn around the cars and labelled with the team name.

Split	Images	Labelled Instances	Background Images
Training	6,657	~7,200	306
Validation	628	660	45
Test	~300	~320	-

Images were resized to whatever resolution each model needed. YOLOv8 uses normalised centre coordinates so the labels from Roboflow could be used directly. For SSD a custom Dataset class had to be written to convert those normalised coordinates to absolute pixel values in the format the model expects.

YOLOv8 Architecture

YOLOv8 is a single-stage, anchor-free object detector developed by Ultralytics. It processes the entire image in one forward pass through a backbone, neck, and detection head. The backbone used here is YOLOv8n (nano), the smallest and fastest variant, chosen to keep training times manageable on a single T4 GPU.

Key architectural features of YOLOv8 include:

- **Anchor-free detection:** Instead of pre-defined anchor boxes, YOLOv8 directly predicts the centre point and dimensions of each object. This reduces the number of candidate predictions and naturally avoids the class imbalance problem common in anchor-based methods.
- **C2f modules:** Cross Stage Partial with 2 feature levels, replacing traditional residual blocks. These allow better gradient flow during backpropagation and more efficient feature reuse.

- PANet neck: Path Aggregation Network that fuses features from multiple scales in both top-down and bottom-up directions, giving the model strong multi-scale detection capability.
- Decoupled head: Separate branches for classification and bounding box regression, which is known to improve convergence speed and accuracy.

The model was fine-tuned from pretrained COCO weights, meaning it already had general visual knowledge before seeing any F1 images. The Ultralytics training framework handles augmentation, learning rate scheduling, and evaluation automatically.

SSD Architecture

SSD (Single Shot MultiBox Detector) is an anchor-based single-stage detector. It was introduced by Liu et al. in 2016 and remains a commonly used baseline in embedded and real-time detection research. The implementation used here is `ssd300_vgg16` from `torchvision`, which uses VGG16 as its backbone and operates at a fixed 300x300 input resolution by default.

Key architectural features of SSD include:

- VGG16 backbone: A deep convolutional network originally designed for image classification, repurposed here for feature extraction. It is heavier than YOLOv8's backbone but benefits from strong pretrained representations.
- Multi-scale anchor boxes: SSD places a grid of default anchor boxes at six different feature map scales, covering objects of different sizes. Each anchor predicts class scores and box offsets simultaneously.
- Smooth L1 loss for regression and softmax cross entropy for classification: standard losses for anchor-based detection.
- No feature pyramid: Unlike YOLOv8's PANet, SSD uses independent feature maps at each scale without cross-scale feature fusion, which limits its ability to detect small objects.

SSD was trained using a custom PyTorch training loop with SGD optimiser. Unlike YOLOv8, SSD does not automatically calculate precision and recall during training, so these were computed separately after training using the `torchmetrics` library.

Training Setup

All experiments were run on Kaggle's free T4 GPU (14.56 GB VRAM). The training platform was chosen over Google Colab because of Kaggle's background execution feature, which allows the session to continue running even if the browser is closed, eliminating the disconnection problem common with Colab.

Both models were fine-tuned from pretrained weights. YOLOv8 used weights pretrained on COCO, while SSD used weights pretrained on COCO via `torchvision`. The optimiser for both was SGD with momentum 0.9 and weight decay 0.0005.

Parameter	YOLOv8	SSD
Backbone	YOLOv8n (CSPNet)	VGG16
Optimiser	AdamW (auto)	SGD
Epochs	50	30
Batch sizes tested	16, 8	16, 8

Parameter	YOLOv8	SSD
Image sizes tested	640, 416	300, 512
Learning rates tested	0.01, 0.001	0.001, 0.0001
Pretrained on	COCO	COCO
Framework	Ultralytics	PyTorch + torchvision

Experimental Configurations

Each model was trained under four configurations, changing one parameter at a time relative to a baseline. This controlled approach allows the effect of each parameter change to be isolated and compared fairly.

Config	Change	YOLOv8 Params	SSD Params
A	Baseline	imgsz=640, batch=16, lr=0.01	imgsz=300, batch=16, lr=0.001
B	Batch size	imgsz=640, batch=8, lr=0.01	imgsz=300, batch=8, lr=0.001
C	Image size	imgsz=416, batch=16, lr=0.01	imgsz=512, batch=16, lr=0.001
D	Learning rate	imgsz=640, batch=16, lr=0.001	imgsz=300, batch=16, lr=0.0001

Evaluation Metrics

The following metrics were used to evaluate model performance:

- mAP50: Mean Average Precision at IoU threshold 0.5. This is the primary metric for object detection and measures how well the model finds and correctly classifies objects when the predicted bounding box overlaps the ground truth by at least 50%.
- mAP50-95: Mean Average Precision averaged across IoU thresholds from 0.5 to 0.95 in steps of 0.05. This is a stricter metric that rewards more precisely fitted bounding boxes.
- Precision: Of all detections made by the model, what fraction were correct. High precision means few false positives.
- Recall: Of all actual objects in the validation set, what fraction the model successfully detected. High recall means few missed detections.
- Per-class AP: The Average Precision calculated separately for each of the four teams, showing which teams are easier or harder to classify.

Experiments and Results

YOLOv8 Results

All four YOLOv8 configurations converged successfully without NaN loss or instability. Training each configuration took approximately 1 hour on the T4 GPU at batch 16, imgsz 640. The Ultralytics framework handled learning rate scheduling automatically, using a cosine schedule with warmup.

Config	Parameters	mAP50	mAP50-95	Precision	Recall	Best Epoch
A (Baseline)	imgsz=640, batch=16, lr=0.01	0.9580	0.7530	0.9220	0.9310	49
B (Batch=8)	imgsz=640, batch=8, lr=0.01	0.9534	0.7284	0.9216	0.9397	39
C (imgsz=416)	imgsz=416, batch=16, lr=0.01	0.9608	0.7611	0.9316	0.9201	44
D (lr=0.001)	imgsz=640, batch=16, lr=0.001	0.9584	0.7531	0.9224	0.9306	49

Config A (baseline) achieved an mAP50 of 0.9580, which is already a very strong result. The model converged slowly, reaching its best performance only at epoch 49 out of 50, suggesting that the full 50 epochs were necessary and that training could potentially have continued further.

Config B reduced the batch size from 16 to 8. This had a slight negative effect on mAP50 (0.9534 vs 0.9580) and mAP50-95 (0.7284 vs 0.7530). Smaller batch sizes produce noisier gradient estimates, which can slow convergence. However, recall improved marginally (0.9397 vs 0.9310), suggesting the model was slightly better at finding all objects even if the overall detection precision dropped. The best epoch was reached earlier at epoch 39, consistent with noisier but faster initial learning.

Config C reduced the image size from 640x640 to 416x416. Somewhat counterintuitively, this was the best performing configuration, reaching an mAP50 of 0.9608 and mAP50-95 of 0.7611, both the highest across all YOLOv8 configs. One possible explanation is that at 416x416, the model was forced to learn more generalised features rather than overfitting to fine pixel-level details. The precision was also highest here (0.9316). The tradeoff was a slightly lower recall (0.9201), meaning more missed detections, but the boxes that were predicted were more reliable.

Config D reduced the learning rate to 0.001. Results were almost identical to Config A (mAP50 0.9584 vs 0.9580), suggesting YOLOv8 with its automatic scheduler is relatively robust to modest learning rate changes. The model still needed 49 epochs to reach its best performance, same as Config A.

YOLOv8 Per-Class Performance

The per-class analysis from Config A (baseline) shows interesting differences between teams:

Team	AP Score	Observation
Red Bull	0.973	Highest AP — distinctive dark blue/yellow livery

Team	AP Score	Observation
Ferrari	0.962	Second highest — iconic red, highly recognisable
McLaren	0.951	Good performance — papaya orange is distinctive
Mercedes	0.934	Lowest AP — silver livery can blend with track/barriers

Mercedes consistently had the lowest per-class AP across all configs. This is likely because its silver livery has lower visual contrast against asphalt, barriers, and certain lighting conditions compared to the more vivid colours of the other three teams. Red Bull having the highest AP is perhaps surprising given its dark livery, but the yellow accents and distinctive sponsor logos likely provide strong discriminative features.

SSD Results

SSD training required more careful handling than YOLOv8. The initial learning rate of 0.005 caused NaN loss due to the focal-loss-like behaviour of the classification head, and had to be reduced to 0.001 before training stabilised. Additionally, SSD showed significant overfitting across all configurations: training loss continued to decrease throughout all 30 epochs while validation loss initially decreased then steadily increased from around epoch 6-7 onwards.

Config	Parameters	mAP50	mAP50-95	Recall@1	Recall@100	Best Val Loss Epoch
A (Baseline)	imgsz=300, batch=16, lr=0.001	0.9131	0.6644	0.7210	0.7507	7
B (Batch=8)	imgsz=300, batch=8, lr=0.001	0.9129	0.6585	0.7226	0.7516	6
C (imgsz=512)	imgsz=512, batch=16, lr=0.001	0.9241	0.6608	0.7175	0.7457	6
D (lr=0.0001)	imgsz=300, batch=16, lr=0.0001	0.8968	0.6451	0.6969	0.7362	17*

Config A (baseline) achieved an mAP50 of 0.9131. While this is technically above 0.9, the best validation loss was achieved as early as epoch 7, after which the model began to overfit. This means the model's peak generalisation performance came very early in training, and the validation loss grew to 2.33 by epoch 30 compared to its best of 1.64. The training loss, by contrast, continued falling all the way to around 0.73 by epoch 30, a clear overfitting signature.

Config B reduced batch size to 8. The results were nearly identical to Config A (mAP50 0.9129 vs 0.9131), with the best validation loss reached even earlier at epoch 6. The smaller batch size did not help with overfitting and arguably made it worse by producing more volatile gradient updates early in training.

Config C increased image size to 512x512. This was the best SSD configuration with an mAP50 of 0.9241. The larger resolution gave the model more detail to work with, particularly helpful for detecting cars at varying distances. The best validation loss was also reached at epoch 6, still showing the same early overfitting pattern, but the peak performance before overfitting was noticeably higher than the 300x300 configs.

Config D reduced the learning rate to 0.0001. This had the most significant negative effect, producing the lowest mAP50 of 0.8968. The very small learning rate slowed convergence considerably, and the training was interrupted at epoch 17 due to a session issue. Despite this, the validation loss reached its best at epoch 17, suggesting the slower learning rate did delay overfitting somewhat, which is consistent with theory. However the overall performance was still the weakest.

SSD Per-Class Performance

Team	SSD-A AP	YOLOv8-A AP	Difference
Red Bull	0.7242	0.9730	-0.2488
McLaren	0.6793	0.9510	-0.2717
Ferrari	0.6280	0.9620	-0.3340
Mercedes	0.6262	0.9340	-0.3078

SSD's per-class performance is notably lower than YOLOv8 across all four teams. The gap is largest for Ferrari and Mercedes, both of which have liveries that can be harder to distinguish in lower resolution or partially occluded conditions. Red Bull performs best for SSD as well, which is consistent with YOLOv8's findings. The consistent ranking (Red Bull > McLaren > Ferrari > Mercedes) across both models suggests this ordering reflects genuine visual difficulty rather than model-specific behaviour.

Cross-Model Comparison

Metric	YOLOv8 Best (Config C)	SSD Best (Config C)	YOLOv8 Avg	SSD Avg
mAP50	0.9608	0.9241	0.9577	0.9117
mAP50-95	0.7611	0.6608	0.7489	0.6572
Recall	0.9201	0.7457	0.9304	0.7461
Inference Speed	5.4ms/frame	~18ms/frame	5.4ms/frame	~18ms/frame

YOLOv8 outperforms SSD on every metric. The mAP50 gap of approximately 4.6 percentage points between averages might seem small, but it represents a meaningful difference in practical detection reliability, especially for fast-moving objects. The mAP50-95 gap is larger (approximately 9 percentage points), indicating that YOLOv8's bounding boxes are substantially more precisely fitted to the actual cars.

The recall gap is the most significant finding from a broadcast application perspective. YOLOv8 achieves an average recall of 0.9304 compared to SSD's 0.7461. In practical terms, this means SSD misses approximately 25% of cars that appear in frame, while YOLOv8 misses less than 7%. For a broadcast use case where every car needs to be tracked, this is a critical difference.

Inference speed is also dramatically different. YOLOv8 processes each frame in approximately 5.4ms, which is well above real-time (25fps = 40ms per frame). SSD, due to its heavier VGG16 backbone and larger anchor

computation overhead, operates at around 18ms per frame, which is still real-time capable but significantly slower. For deployment on less powerful hardware, this gap would widen further.

Discussion

Why YOLOv8 Outperforms SSD

There are a few reasons why YOLOv8 came out so much better. The biggest one is probably the anchor-free design. SSD generates thousands of anchor boxes across the image and most of them are background. This creates a big class imbalance during training that is hard to deal with properly. YOLOv8 avoids this entirely by predicting object locations directly without anchors.

The way YOLOv8 handles multi-scale features also helps. SSD uses separate feature maps at different scales but they do not share information with each other. YOLOv8's PANet goes both top-down and bottom-up, so fine-grained detail and high-level semantic understanding get combined. This probably makes a difference when detecting small or distant cars.

VGG16 is also a much bigger network than YOLOv8n. It has around 138 million parameters compared to YOLOv8n's 3 million. For a dataset of this size that is probably too much capacity and explains why SSD overfit so heavily.

The Overfitting Problem in SSD

The overfitting in SSD was one of the most notable things from the experiments. The validation loss bottomed out around epoch 6 to 7 in most configs, then just kept climbing while training loss went down. By epoch 30 the gap between them was quite large. The model was basically learning the training images by heart rather than picking up on features that would transfer to new images.

Two things probably caused this. First, VGG16 is just too big for this dataset. 138 million parameters for 6,600 training images is a lot, and the model has more than enough capacity to memorise everything. Second, the Ultralytics framework that YOLOv8 uses applies a lot of augmentation automatically during training, things like mosaic, colour jitter, and flipping. The custom SSD training loop did not have any of that, which means there was no augmentation acting as a regulariser.

The lesson here is that architecture choice and training infrastructure matter as much as the model itself. A well-tuned training pipeline with appropriate regularisation could likely close some of the gap between SSD and YOLOv8 on this task.

Impact of Hyperparameter Choices

The hyperparameter experiments revealed different sensitivities for each model. YOLOv8 was relatively robust across all four configurations, with mAP50 ranging only from 0.9534 to 0.9608. The image size reduction to 416 (Config C) surprisingly gave the best result, which may indicate that at the original 640 resolution, the model was overfitting to high-frequency texture details that do not generalise well. The smaller resolution forced the model to rely on more abstract shape and colour features.

SSD showed more sensitivity to parameter changes, particularly learning rate. Config D with $lr=0.0001$ had the worst performance, suggesting that with only 30 epochs and such a small learning rate, the model did not have enough updates to fully adapt its pretrained weights to the F1 domain.

Implications for F1 Broadcasting

The video test demonstrated that YOLOv8 can successfully detect and classify F1 cars in real broadcast footage. The model correctly identified cars from all four teams across 668 frames, with an average inference speed of

5.4ms per frame. At 25fps, each frame has 40ms available for processing, meaning the model comfortably meets real-time requirements even with overhead for other processing.

Potential broadcast applications include automated graphics overlays that label each car on screen as it appears, statistical tracking of on-screen time for each team, automated highlights generation where specific team appearances trigger clip extraction, and real-time telemetry synchronisation where the car's visual position is used to anchor data feeds.

However, there are limitations. The model struggled with certain frames, particularly when cars were partially out of frame, heavily motion-blurred, or shot from unusual angles. The dataset, while large, may not cover all the visual conditions encountered in a full season of racing. Further training with more diverse footage, including wet weather conditions, night races, and different broadcast camera angles, would be needed for production deployment.

Conclusion

The main takeaway from this project is that YOLOv8 is clearly the better option for this kind of task. It scored higher on every metric, was faster, and was much more stable during training. The gap was larger than expected going in, particularly on recall.

The best result was from YOLOv8 Config C at 0.9608 mAP50, and the model worked well on actual F1 footage, picking out cars from all four teams consistently. SSD managed above 0.9 mAP50 in its best config which is decent, but the overfitting and the much lower recall (around 0.75 compared to YOLOv8's 0.93) make it less practical for anything where you cannot afford to miss detections.

One interesting result was that reducing the image size for YOLOv8 from 640 to 416 actually slightly improved performance, which was not what I expected. For SSD it was the opposite, the larger 512 resolution gave the best results. Both models were fairly stable across learning rate changes, except SSD at $lr=0.0001$ which was too slow to converge properly in 30 epochs.

If I were to continue this work, the first thing to try would be adding proper augmentation to the SSD training loop to see if that fixes the overfitting. Testing on more varied footage like wet races or night events would also give a better picture of how robust the model really is. It would also be worth trying a larger YOLOv8 variant to see how much the accuracy could improve, though inference speed would need to be considered for a real broadcast setup.

References

- Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 580-587.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. In European Conference on Computer Vision (ECCV), 21-37. Springer.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>
- Kamble, P., Keskar, A., & Bhurchandi, K. (2019). A ball tracking system for sports applications. Artificial Intelligence Review, 52(1), 541-572.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In International Conference on Learning Representations (ICLR).
- F1 Car Classification Dataset. (2023). Roboflow Universe. <https://universe.roboflow.com/f1-car-detection-model/f1-car-classification>
- Lin, T. Y., Dollar, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117-2125.
- Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. arXiv:2004.10934.